

# FinAgent

---

## Plano de Implantação Técnica — Stack Django

Requisitos • Infraestrutura • Arquitetura • Cronograma • Deploy

Do desenvolvimento local ao deploy em produção via VPS

UFG — Especialização em Sistemas e Agentes Inteligentes  
Abril 2026 — Versão 2.0 (Django)

# 1. Por Que Django para o FinAgent

A disciplina de Agentes Inteligentes da UFG utiliza Django como framework web. Além de atender ao requisito acadêmico, Django traz vantagens concretas para o projeto:

- **ORM nativo robusto:** models.py define as tabelas e Django cuida de migrations, queries e relacionamentos. Ideal para o modelo de dados contábil hierarquizado.
- **Admin Panel gratuito:** registrando os models no admin.py, você ganha uma interface completa para visualizar, filtrar e editar balancetes, indicadores e scores sem escrever uma linha de frontend.
- **Autenticação pronta:** login, logout, permissões — tudo nativo. Importante se a banca perguntar sobre segurança.
- **Arquitetura MTV escalável:** Model-Template-View separa bem as responsabilidades e facilita a organização dos agentes.
- **Ecossistema maduro:** Django REST Framework se precisar de API, Celery para tarefas em background, django-plotly-dash para dashboards.
- **Alinhamento acadêmico:** demonstra domínio do framework usado na disciplina.

## 1.1 Comparação com Streamlit

| Critério                   | Django                      | Streamlit                        |
|----------------------------|-----------------------------|----------------------------------|
| Requisito da disciplina    | ✓ Sim                       | ✗ Não                            |
| ORM / Banco de dados       | ✓ Nativo (poderoso)         | ✗ Precisa de SQLAlchemy separado |
| Admin Panel                | ✓ Gratuito e completo       | ✗ Não existe                     |
| Autenticação               | ✓ Nativa                    | ✗ Precisa de lib externa         |
| Frontend flexível          | ✓ Templates + JS livre      | ⚠ Limitado aos widgets nativos   |
| Dashboards interativos     | ⚠ Plotly via template (bom) | ✓ Nativo e rápido                |
| Velocidade de prototipação | ⚠ Média                     | ✓ Muito rápida                   |
| Escalabilidade             | ✓ Alta                      | ✗ Baixa                          |
| Deploy profissional        | ✓ Gunicorn + Nginx          | ⚠ Streamlit Cloud (limitado)     |

## 2. Stack Tecnológica Definitiva

### 2.1 Backend

| Camada        | Tecnologia              | Versão        | Justificativa                                             |
|---------------|-------------------------|---------------|-----------------------------------------------------------|
| Framework Web | Django                  | 5.1+          | Requisito da disciplina, ORM, admin, auth, MTV            |
| Linguagem     | Python                  | 3.11+         | Ecossistema data science + Django nativo                  |
| WSGI Server   | Gunicorn                | 22+           | Servidor de produção para Django                          |
| Task Queue    | Celery + Redis          | 5.4+          | Processamento em background dos agentes (pipeline pesado) |
| Processamento | Pandas + NumPy          | 2.2+ / 1.26+  | Manipulação de DataFrames contábeis                       |
| Validação     | Django Forms + Pydantic | Nativo / 2.7+ | Validação de upload + schemas dos agentes                 |

### 2.2 Inteligência Artificial

| Camada                  | Tecnologia          | Uso                                                               |
|-------------------------|---------------------|-------------------------------------------------------------------|
| Orquestração de Agentes | CrewAI              | Pipeline multi-agente (ou orquestração manual com classes Python) |
| LLM Principal           | Claude API (Sonnet) | Risk Scorer (narrativas) + Chat Agent (NLP)                       |
| LLM Leve                | Claude Haiku        | Classificação de contas, tarefas simples                          |
| ML / Forecast           | Prophet             | Projeções de séries temporais                                     |
| SDK                     | anthropic (Python)  | Comunicação com a API Claude                                      |

### 2.3 Banco de Dados

| Fase                   | Tecnologia    | Justificativa                                              |
|------------------------|---------------|------------------------------------------------------------|
| Desenvolvimento        | SQLite        | Zero config, já vem no Django, perfeito para dev local     |
| Produção (VPS)         | PostgreSQL 16 | Robusto, concorrência, Django tem suporte nativo excelente |
| Cache / Message Broker | Redis         | Cache de respostas da API + broker do Celery               |

### 2.4 Frontend (Django Templates)

| Componente | Tecnologia                    | Uso                                         |
|------------|-------------------------------|---------------------------------------------|
| Templates  | Django Templates (Jinja-like) | HTML com lógica de renderização server-side |

| Componente           | Tecnologia                  | Uso                                                             |
|----------------------|-----------------------------|-----------------------------------------------------------------|
| CSS Framework        | Bootstrap 5 ou Tailwind CSS | Layout responsivo, componentes prontos (cards, navbars, modais) |
| Gráficos Interativos | Plotly.js                   | Renderização client-side via JSON gerado no Django              |
| Interatividade       | HTMX (opcional)             | Atualização parcial de página sem recarregar                    |
| Ícones               | Bootstrap Icons ou Lucide   | Iconografia para dashboards                                     |

## 2.5 DevOps

| Componente         | Tecnologia              | Uso                                                          |
|--------------------|-------------------------|--------------------------------------------------------------|
| Controle de versão | Git + GitHub            | Código versionado                                            |
| Deploy             | Docker + Docker Compose | Container com Django + Gunicorn + Nginx + PostgreSQL + Redis |
| Proxy Reverso      | Nginx                   | Serve static files + proxy para Gunicorn                     |
| SSL                | Certbot (Let's Encrypt) | HTTPS gratuito                                               |
| Static Files       | WhiteNoise ou Nginx     | Servir CSS/JS/imagens em produção                            |

### 3. Estrutura do Projeto Django

O projeto segue o padrão Django com apps modulares. Cada domínio funcional é uma app separada.

```

finagent/
├── manage.py
├── finagent/                                # Projeto Django (settings, urls, wsgi)
│   ├── __init__.py
│   ├── settings.py                        # Configurações (DB, apps, middleware)
│   ├── urls.py                           # URLs raiz
│   ├── wsgi.py                           # Entry point WSGI (Gunicorn)
│   ├── asgi.py                           # Entry point ASGI (opcional)
│   └── celery.py                         # Configuração do Celery
├── core/                                  # App: infraestrutura compartilhada
│   ├── models.py                         # BaseModel com created_at, updated_at
│   ├── llm_client.py                     # Wrapper da API Claude
│   ├── utils.py                          # Funções utilitárias
│   └── templatetags/
│       └── format_tags.py                # Filtros (formatar moeda, %)
├── empresas/                             # App: cadastro de empresas
│   ├── models.py                         # Empresa
│   ├── admin.py
│   ├── views.py
│   ├── urls.py
│   └── forms.py
├── contabil/                             # App: dados contábeis
│   ├── models.py                         # Período, PlanoContas, Balancete
│   ├── admin.py                         # Admin com filtros por empresa/período
│   ├── views.py                         # Upload, visualização do balancete
│   ├── urls.py
│   ├── forms.py
│   └── services/
│       └── parser_service.py             # Form de upload de arquivo
│                                           # Lógica de negócio (fora das views)
│                                           # Lógica do Agente 1 (Parser)
├── analises/                             # App: DRE, análises V&H, indicadores
│   ├── models.py                         # DRELinha, AnaliseVertical,
│   └── admin.py                         AnaliseHorizontal, Indicador
│   ├── views.py                         # Views com gráficos Plotly
│   ├── urls.py
│   └── services/
│       ├── dre_service.py                # Agente 2 (DRE Builder)
│       ├── analyzer_service.py           # Agente 3 (Análises V&H)
│       └── kpi_service.py                 # Agente 4 (KPI Engine)
├── risco/                                 # App: score de risco
│   ├── models.py                         # Score
│   ├── admin.py
│   ├── views.py                         # Dashboard de risco + radar chart
│   ├── urls.py
│   └── services/
│       └── risk_service.py                # Agente 5 (Risk Scorer + Claude API)

```

|                             |                                           |
|-----------------------------|-------------------------------------------|
| └─ forecast/                | # App: projeções ML                       |
| └─ models.py                | # Projecao                                |
| └─ views.py                 | # Gráficos de forecast                    |
| └─ urls.py                  |                                           |
| └─ services/                |                                           |
| └─ forecast_service.py      | # Agente 6 (Prophet)                      |
| └─ chat/                    | # App: chat NLP                           |
| └─ models.py                | # ChatHistorico                           |
| └─ views.py                 | # Interface de chat (AJAX)                |
| └─ urls.py                  |                                           |
| └─ services/                |                                           |
| └─ chat_service.py          | # Agente 7 (Chat + Claude API)            |
| └─ agents/                  | # Orquestração dos agentes                |
| └─ base_agent.py            | # Classe base abstrata                    |
| └─ orchestrator.py          | # Pipeline sequencial                     |
| └─ tasks.py                 | # Celery tasks (processamento async)      |
| └─ templates/               | # Templates HTML                          |
| └─ base.html                | # Layout master (navbar, sidebar, footer) |
| └─ home.html                | # Home / Dashboard geral                  |
| └─ contabil/                |                                           |
| └─ upload.html              | # Upload de balancete                     |
| └─ balancete_detail.html    | # Visualização do balancete               |
| └─ analises/                |                                           |
| └─ dre.html                 | # DRE Gerencial                           |
| └─ vertical_horizontal.html | # Análises V&H                            |
| └─ indicadores.html         | # Dashboard de KPIs                       |
| └─ risco/                   |                                           |
| └─ score.html               | # Score + radar + narrativa IA            |
| └─ forecast/                |                                           |
| └─ projecoes.html           | # Gráficos de forecast                    |
| └─ chat/                    |                                           |
| └─ chat.html                | # Interface de chat                       |
| └─ static/                  | # Arquivos estáticos                      |
| └─ css/                     |                                           |
| └─ style.css                | # Estilos customizados                    |
| └─ js/                      |                                           |
| └─ charts.js                | # Funções Plotly reutilizáveis            |
| └─ chat.js                  | # Lógica AJAX do chat                     |
| └─ img/                     |                                           |
| └─ data/                    | # Dados de exemplo                        |
| └─ sample_balancete.xlsx    |                                           |
| └─ plano_contas_padrao.csv  |                                           |
| └─ deploy/                  | # Configurações de deploy                 |
| └─ Dockerfile               |                                           |
| └─ docker-compose.yml       |                                           |
| └─ nginx.conf               |                                           |
| └─ entrypoint.sh            |                                           |
| └─ tests/                   |                                           |
| └─ .env.example             |                                           |
| └─ .gitignore               |                                           |

```
|— requirements.txt  
|— README.md
```

## 4. Modelos Django (Banco de Dados)

Os modelos seguem o padrão Django e definem toda a arquitetura de dados. O ORM cuida das migrations automaticamente.

### 4.1 core/models.py — Base

```
class BaseModel(models.Model):
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    class Meta:
        abstract = True
```

### 4.2 empresas/models.py

```
class Empresa(BaseModel):
    nome = models.CharField(max_length=200)
    cnpj = models.CharField(max_length=18, unique=True, blank=True, null=True)
    setor = models.CharField(max_length=100, blank=True)
    regime_tributario = models.CharField(max_length=50, blank=True)
```

### 4.3 contabil/models.py

```
class Periodo(BaseModel):
    empresa = models.ForeignKey(Empresa, on_delete=models.CASCADE)
    ano = models.IntegerField()
    mes = models.IntegerField(validators=[MinValue(1), MaxValue(12)])
    tipo = models.CharField(choices=[('mensal', 'Mensal'), ('trim', 'Trimestral')])
    arquivo_origem = models.FileField(upload_to='balancetes/')
    status = models.CharField(default='pendente') # pendente/processado/erro
```

```
class PlanoContas(BaseModel):
    empresa = models.ForeignKey(Empresa, on_delete=models.CASCADE)
    codigo = models.CharField(max_length=20)
    descricao = models.CharField(max_length=300)
    grupo = models.CharField() # ATIVO/PASSIVO/RECEITA/DESPESA/CUSTO
    subgrupo = models.CharField(blank=True)
    nivel = models.IntegerField()
    natureza = models.CharField(max_length=1) # D ou C
    tipo_conta = models.CharField(max_length=1) # S ou A
```

```
class Balancete(BaseModel):
    periodo = models.ForeignKey(Periodo, on_delete=models.CASCADE)
    conta = models.ForeignKey(PlanoContas, on_delete=models.CASCADE)
    saldo_anterior = models.DecimalField(max_digits=15, decimal_places=2)
    debito = models.DecimalField(max_digits=15, decimal_places=2)
    credito = models.DecimalField(max_digits=15, decimal_places=2)
    saldo_atual = models.DecimalField(max_digits=15, decimal_places=2)
```

### 4.4 analises/models.py

```
class DRELinha(BaseModel):
    periodo = models.ForeignKey(Periodo, on_delete=models.CASCADE)
    ordem = models.IntegerField()
    descricao = models.CharField(max_length=200)
```



```
valor = models.DecimalField(max_digits=15, decimal_places=2)
percentual_av = models.DecimalField(max_digits=8, decimal_places=4)

class Indicador(BaseModel):
    periodo = models.ForeignKey(Periodo, on_delete=models.CASCADE)
    categoria = models.CharField() #
    liquidez/rentabilidade/endividamento/atividade
    nome = models.CharField(max_length=100)
    valor = models.DecimalField(max_digits=12, decimal_places=4)
    referencia_setor = models.DecimalField(null=True, blank=True)
    status = models.CharField() # bom/atencao/critico
```

## 4.5 risco/models.py

```
class Score(BaseModel):
    periodo = models.OneToOneField(Periodo, on_delete=models.CASCADE)
    score_geral = models.DecimalField(max_digits=5, decimal_places=2)
    score_liquidez = models.DecimalField(max_digits=5, decimal_places=2)
    score_rentabilidade = models.DecimalField(max_digits=5, decimal_places=2)
    score_endividamento = models.DecimalField(max_digits=5, decimal_places=2)
    score_operacional = models.DecimalField(max_digits=5, decimal_places=2)
    narrativa_ia = models.TextField(blank=True) # Gerada pelo Claude
    classificacao = models.CharField() # excelente/boa/atencao/risco/critico
```

## 5. Padrão de Views com Gráficos Plotly

A integração Django + Plotly funciona assim: a view gera o gráfico em Python, converte para HTML e passa ao template.

### 5.1 View Exemplo (indicadores)

```
# analises/views.py
import plotly.graph_objects as go
from plotly.utils import PlotlyJSONEncoder
import json

def dashboard_indicadores(request, periodo_id):
    indicadores = Indicador.objects.filter(periodo_id=periodo_id)

    # Gerar gráfico Plotly
    fig = go.Figure(go.Indicator(
        mode='gauge+number',
        value=indicadores.filter(nome='Liquidez Corrente').first().valor,
        title={'text': 'Liquidez Corrente'},
        gauge={'axis': {'range': [0, 3]}}
    ))
    chart_json = json.dumps(fig, cls=PlotlyJSONEncoder)

    return render(request, 'analises/indicadores.html', {
        'indicadores': indicadores,
        'chart_json': chart_json,
    })
```

### 5.2 Template Exemplo

```
<!-- templates/analises/indicadores.html -->
{% extends 'base.html' %}
{% block content %}
<div id="chart"></div>
<script src="https://cdn.plot.ly/plotly-latest.min.js"></script>
<script>
    var data = {{ chart_json|safe }};
    Plotly.newPlot('chart', data.data, data.layout);
</script>
{% endblock %}
```

### 5.3 Padrão do Chat via AJAX

```
# chat/views.py
from django.http import JsonResponse
from .services.chat_service import ChatAgent

def chat_api(request):
    if request.method == 'POST':
        pergunta = json.loads(request.body)['pergunta']
        agent = ChatAgent(empresa_id=request.session['empresa_id'])
        resposta = agent.responder(pergunta)
        return JsonResponse({'resposta': resposta})
```

O frontend do chat usa JavaScript puro (fetch API) para enviar perguntas e receber respostas sem recarregar a página.

## 6. Pipeline dos Agentes no Django

Cada agente é implementado como um service dentro da app correspondente. O orchestrator coordena a execução sequencial.

### 6.1 Classe Base do Agente

```
# agents/base_agent.py
from abc import ABC, abstractmethod
import logging

class BaseAgent(ABC):
    def __init__(self, periodo_id):
        self.periodo_id = periodo_id
        self.logger = logging.getLogger(self.__class__.__name__)

    @abstractmethod
    def executar(self):
        """Executa a lógica do agente e salva resultados no DB."""
        pass

    def validar_inputs(self):
        """Verifica se os dados necessários existem."""
        pass
```

### 6.2 Orquestrador

```
# agents/orchestrator.py
class FinAgentOrchestrator:
    def __init__(self, periodo_id):
        self.periodo_id = periodo_id

    def executar_pipeline(self):
        """Executa todos os agentes em sequência."""
        agents = [
            ParserService(self.periodo_id),      # Agente 1
            DREService(self.periodo_id),         # Agente 2
            AnalyzerService(self.periodo_id),    # Agente 3
            KPIService(self.periodo_id),         # Agente 4
            RiskService(self.periodo_id),        # Agente 5
        ]
        for agent in agents:
            agent.validar_inputs()
            agent.executar()
        # Agentes 6 e 7 são sob demanda
```

### 6.3 Celery Task (Background)

```
# agents/tasks.py
from celery import shared_task

@shared_task
def processar_balancete(periodo_id):
```

```
orchestrator = FinAgentOrchestrator(periodo_id)
orchestrator.executar_pipeline()
Periodo.objects.filter(id=periodo_id).update(status='processado')
```

O usuário faz upload, o Celery processa em background, e a página atualiza o status via polling ou HTMX.

## 7. Mapa de URLs e Navegação

| URL                        | View             | Template                          | Descrição                             |
|----------------------------|------------------|-----------------------------------|---------------------------------------|
| /                          | home             | home.html                         | Dashboard geral com resumo            |
| /empresas/                 | empresa_list     | empresa_list.html                 | Lista de empresas cadastradas         |
| /empresas/nova/            | empresa_create   | empresa_form.html                 | Cadastrar nova empresa                |
| /upload/                   | upload_balancete | contabil/upload.html              | Upload de arquivo Excel/CSV           |
| /balancete/<id>/           | balancete_detail | contabil/balancete_detail.html    | Visualização do balancete normalizado |
| /dre/<periodo_id>/         | dre_view         | analises/dre.html                 | DRE Gerencial + waterfall chart       |
| /analises/<periodo_id>/    | analises_vh      | analises/vertical_horizontal.html | Análises Vertical e Horizontal        |
| /indicadores/<periodo_id>/ | indicadores      | analises/indicadores.html         | Dashboard de KPIs (gauge charts)      |
| /risco/<periodo_id>/       | score_risco      | risco/score.html                  | Score de risco + radar + narrativa IA |
| /projecoes/<periodo_id>/   | projecoes        | forecast/projecoes.html           | Gráficos de forecast (Prophet)        |
| /chat/                     | chat_view        | chat/chat.html                    | Interface de chat NLP                 |
| /api/chat/                 | chat_api         | — (JSON)                          | Endpoint AJAX para o chat             |
| /admin/                    | Django Admin     | — (nativo)                        | Admin panel (visualizar dados)        |

## 8. Dependências (requirements.txt)

```
# Framework
django>=5.1
gunicorn>=22.0
whitenoise>=6.7           # Static files em produção

# Banco de dados
psycopg2-binary>=2.9      # Driver PostgreSQL (produção)

# Tasks em background
celery>=5.4
redis>=5.0                 # Broker do Celery

# Processamento de dados
pandas>=2.2
numpy>=1.26
```

```
openpyxl>=3.1

# IA / LLM
anthropic>=0.40

# Machine Learning
prophet>=1.1

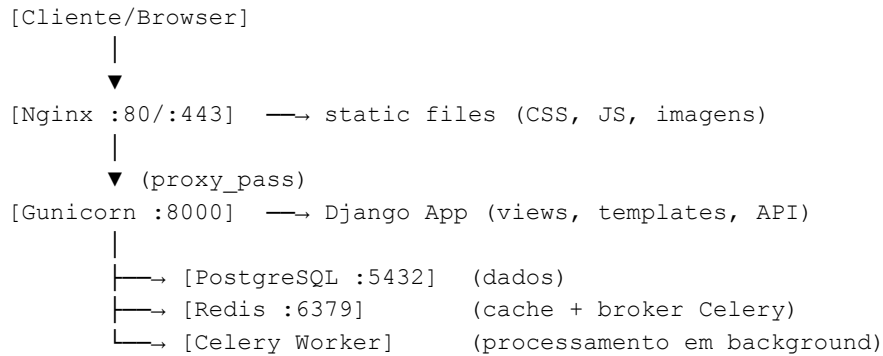
# Visualização
plotly>=5.22

# Utilitários
python-dotenv>=1.0
pydantic>=2.7

# Dev/Testes
pytest-django>=4.8
django-debug-toolbar>=4.4
```

## 9. Deploy em Produção (VPS)

### 9.1 Arquitetura de Deploy



### 9.2 docker-compose.yml

```

version: '3.8'
services:
  web:
    build: .
    command: gunicorn finagent.wsgi:application --bind 0.0.0.0:8000
    volumes:
      - static_volume:/app/staticfiles
      - media_volume:/app/media
    env_file: .env
    depends_on:
      - db
      - redis

  db:
    image: postgres:16
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      POSTGRES_DB: finagent
      POSTGRES_USER: finagent
      POSTGRES_PASSWORD: ${DB_PASSWORD}

  redis:
    image: redis:7-alpine

  celery:
    build: .
    command: celery -A finagent worker -l info
    env_file: .env
    depends_on:
      - db
      - redis

  nginx:
    image: nginx:alpine
    ports:

```



```
- '80:80'
- '443:443'
volumes:
- ./deploy/nginx.conf:/etc/nginx/conf.d/default.conf
- static_volume:/app/staticfiles
depends_on:
- web

volumes:
  postgres_data:
  static_volume:
  media_volume:
```

## 9.3 Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
EXPOSE 8000
```

## 9.4 Variáveis de Ambiente (.env)

```
# Django
SECRET_KEY=sua-chave-secreta-aqui
DEBUG=False
ALLOWED_HOSTS=finagent.seudominio.com.br

# Banco de Dados
DB_ENGINE=django.db.backends.postgresql
DB_NAME=finagent
DB_USER=finagent
DB_PASSWORD=senha-segura
DB_HOST=db
DB_PORT=5432

# Redis / Celery
CELERY_BROKER_URL=redis://redis:6379/0

# Claude API
ANTHROPIC_API_KEY=sk-ant-xxxxx
CLAUDE_MODEL=claude-sonnet-4-20250514
```

## 10. Cronograma Detalhado (8 Semanas)

| Semana | Foco                            | Entregas                                                                                                                           | Horas   |
|--------|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------|---------|
| 1      | Setup Django + Models           | Projeto criado, apps registradas, models definidos, migrations rodando, admin funcional, base.html com Bootstrap                   | 15-20 h |
| 2      | Agente 1 (Parser) + Upload      | Upload de Excel/CSV via form, parser processando e salvando no DB, plano de contas classificado, tela de visualização do balancete | 15-20 h |
| 3      | Agentes 2-3 (DRE + Análises)    | DRE Gerencial montado, análises V&H calculadas, gráficos Plotly (waterfall DRE, barras comparação), 3 templates funcionando        | 15-20 h |
| 4      | Agente 4 (KPIs) + Dashboard     | Todos indicadores calculados (liquidez, rentabilidade, endividamento, atividade), dashboard com gauge charts e cards coloridos     | 15-20 h |
| 5      | Agente 5 (Risk Scorer) + Claude | Score ponderado calculado, integração Claude API gerando narrativas, radar chart por dimensão, tela de risco completa              | 15-20 h |
| 6      | Agentes 6-7 (Forecast + Chat)   | Prophet rodando projeções, chat funcional via AJAX + Claude, Celery processando em background                                      | 15-20 h |
| 7      | Testes + Refinamento            | Testar com 3-5 balancetes reais, corrigir bugs, polir UX, documentação (README, docstrings)                                        | 10-15 h |
| 8      | Deploy + Apresentação           | Docker + VPS configurados, HTTPS, domínio, slides da apresentação para a banca da UFG                                              | 10-15 h |

**Total estimado:** 110-155 horas ao longo de 8 semanas (~14-19h/semana)

## 11. Comandos Iniciais para Começar

Sequência exata de comandos para criar o projeto do zero:

```
# 1. Criar e ativar ambiente virtual
python -m venv .venv
source .venv/bin/activate

# 2. Instalar Django e dependências básicas
pip install django pandas openpyxl plotly anthropic python-dotenv

# 3. Criar projeto Django
django-admin startproject finagent .

# 4. Criar as apps
python manage.py startapp core
python manage.py startapp empresas
python manage.py startapp contabil
python manage.py startapp analises
python manage.py startapp risco
python manage.py startapp forecast
```

```
python manage.py startapp chat

# 5. Registrar apps no settings.py (INSTALLED_APPS)

# 6. Definir models e rodar migrations
python manage.py makemigrations
python manage.py migrate

# 7. Criar superusuário para o admin
python manage.py createsuperuser

# 8. Rodar o servidor
python manage.py runserver

# Acesse: http://127.0.0.1:8000
# Admin:  http://127.0.0.1:8000/admin
```

## 12. Estimativa de Custos

| Fase            | Item                         | Custo/mês  | Obs                           |
|-----------------|------------------------------|------------|-------------------------------|
| Desenvolvimento | Python, Django, Git, VS Code | R\$ 0      | Tudo open-source              |
| Desenvolvimento | Claude API                   | R\$ 25-75  | ~US\$ 5-15 no desenvolvimento |
| Produção        | VPS (Hostinger/Contabo)      | R\$ 25-50  | 2-4 vCPU, 4-8 GB RAM          |
| Produção        | Domínio .com.br              | R\$ 3-5    | R\$ 40-60/ano                 |
| Produção        | SSL (Let's Encrypt)          | R\$ 0      | Gratuito                      |
| Produção        | Claude API (uso contínuo)    | R\$ 50-150 | Depende do volume             |

**Custo total do MVP (2 meses):** R\$ 100-300 (basicamente API Claude + VPS no último mês)

## 13. Decisões Simplificadoras para o MVP

Para caber no prazo de 2 meses com Python básico, as seguintes simplificações são recomendadas:

- **SQLite no desenvolvimento:** trocar para PostgreSQL só no deploy. O ORM do Django abstrai, então é só mudar o settings.py.
- **Celery é opcional no início:** pode processar os agentes de forma síncrona (na própria view) e adicionar Celery só se o processamento ficar lento.
- **HTMX é opcional:** começar com reload completo da página. Adicionar HTMX depois para suavizar a UX.
- **CrewAI é opcional:** a orquestração pode ser feita com classes Python puras (base\_agent + orchestrator). CrewAI adiciona complexidade.
- **Forecast:** só implementar se tiver dados históricos suficientes (12+ meses). Se não tiver, mostrar a arquitetura na apresentação.
- **Frontend simples:** Bootstrap 5 com templates básicos. Não investir tempo em design customizado — a banca quer ver os agentes funcionando.